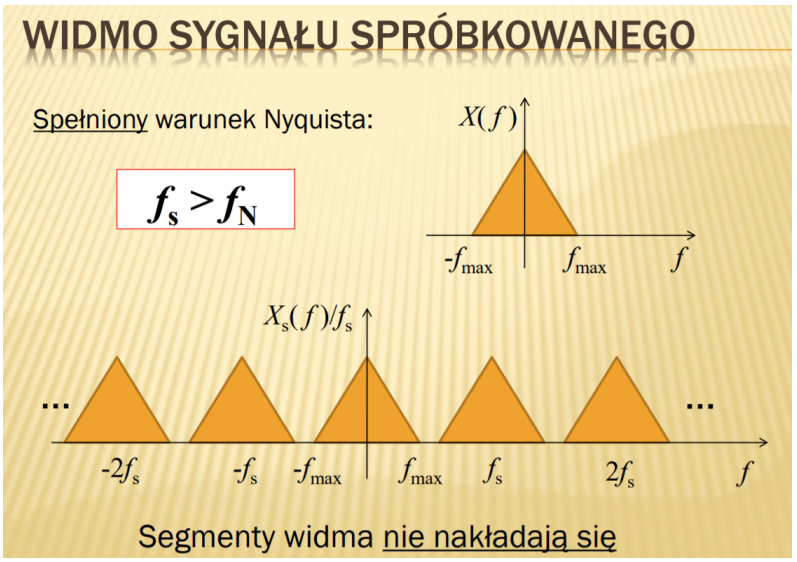
1. **Twierdzenie o próbkowaniu -**

Częstotliwość próbkowania fp musi być większa niż dwukrotność najwyższej składowej częstotliwości fs w mierzonym sygnale.
fp > 2*fs

Próbkowanie – proces tworzenia sygnału dyskretnego, reprezentującego sygnał ciągły za pomocą ciągu wartości nazywanych **próbkami**. Zwykle jest jednym z etapów przetwarzania sygnału analogowego na sygnał cyfrowy.
Okres próbkowania Ts to czas pomiędzy pobieraniem kolejnych próbek.

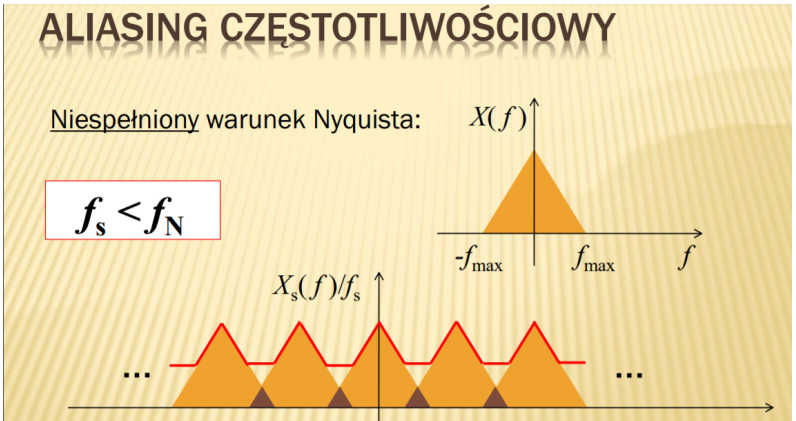
Aby spróbkowany sygnał z postaci dyskretnej dało się przekształcić bez straty informacji z powrotem do postaci ciągłej, musi być spełniony warunek twierdzenia Kotielnikowa-Shannona o próbkowaniu (tak zwany warunek Nyquista). Mówi on, że częstotliwość próbkowania nie może być mniejsza niż podwojona szerokość pasma sygnału przed spróbkowaniem.

Jeśli ten warunek nie jest spełniony, wówczas występuje zjawisko **aliasingu-poszczególne segmenty widma nachodzą na siebie. Niemożliwe jest odtworzenie sygnału analogowego z ciągu próbek.** informacja zawarta w oryginalnym sygnale (analogowym) może nie być w całości odtworzona na podstawie sygnału dyskretnego (nieodwracalne **zniekształcenie** sygnału).
Kwantyzacja – przypisanie danej próbki do określonego poziomu kwantyzacji
https://www.youtube.com/watch?v=U8zegBXvSP8&feature=youtu.be&t=857



PS.Sygnał dyskretny ma widmo okresowe składające się z kopii widma sygnału analogowego scentrowanych wokół wielokrotności częstotliwości próbkowania. (to wiele tych trójkątów)

w tym wypadku:
fs-cz.próbkowania
fN- cz. Nyquista równa 2f_max



2. **Wektory przerwań -** są umieszczone w pamięci **flash** między adresami 0x0000 a 0x0030
.http://www.if.pw.edu.pl/~agatka/psm/wyklad_03.pdf tu fajnie opisane

przerwanie jest zdarzeniem, które przerywa wykonywanie programu głównego, zapamiętuje adres powrotu do programu i uruchamia specjalną funkcję obsługi przerwania. Gdy funkcja obsłuży przerwanie, następuje powrót do przerwanego programu głównego i wznowienie jego wykonywania od miejsca, w którym został przerwany.
Przerwania mogą być **wewnętrzne** (tzn. pochodzić od wewnętrznych składników mikrokontrolera) lub **zewnętrzne** (tzn. generowane przez urządzenie zewnętrzne, którym steruje mikrokontroler).
Przerwanie **sprzętowe** zachodzi w wyniku zmiany stanu jednego z układów fizycznych. Przerwanie **programowe** zachodzi w wyniku wykonania instrukcji programu

• ATMega16 reaguje na 21 różnych rodzajów przerwań(tabela wektorów)
• Przerwania są numerowane zgodnie z ich priorytetem od 1 do 21
• Przerwanie RESET ma numer 1
• Każde przerwanie powoduje skok do specyficznego dla niego adresu w pamięci programu,np. Procedura przerwania RESET ma adres $0000
Ponieważ jest miejsce tylko na jedną lub dwie instrukcje, procedura przerwania rozpoczyna się od skoku w inne miejsce pamięci programu, gdzie znajduje się reszta kodu.

Działanie przerwań
• Jeżeli:
› Globalny bit przerwań jest ustawiony (SREG i=1)
› ORAZ bit konkretnego przerwania jest ustawiony
› ORAZ zaszedł warunek przerwania
• To wystąpi przerwanie

• Co się właściwie dzieje?
› Po zakończeniu aktualnej instrukcji,
• Aktualna wartość licznika programu PC odkładana jest na stos
• Kasowany jest globalny bit przerwań (SREG i=0 zablokowanie)
• Właściwy adres procedury obsługi przerwania jest umieszczany w liczniku programu- Instrukcja w wektorze przerwań wykonuje skok (rjmp = relative jump, czyli skok względny) do odpowiedniej

procedury obsługi przerwania, zwanej ISR (ang. Interrupt Service Routine), która wykonuje pożądane działania

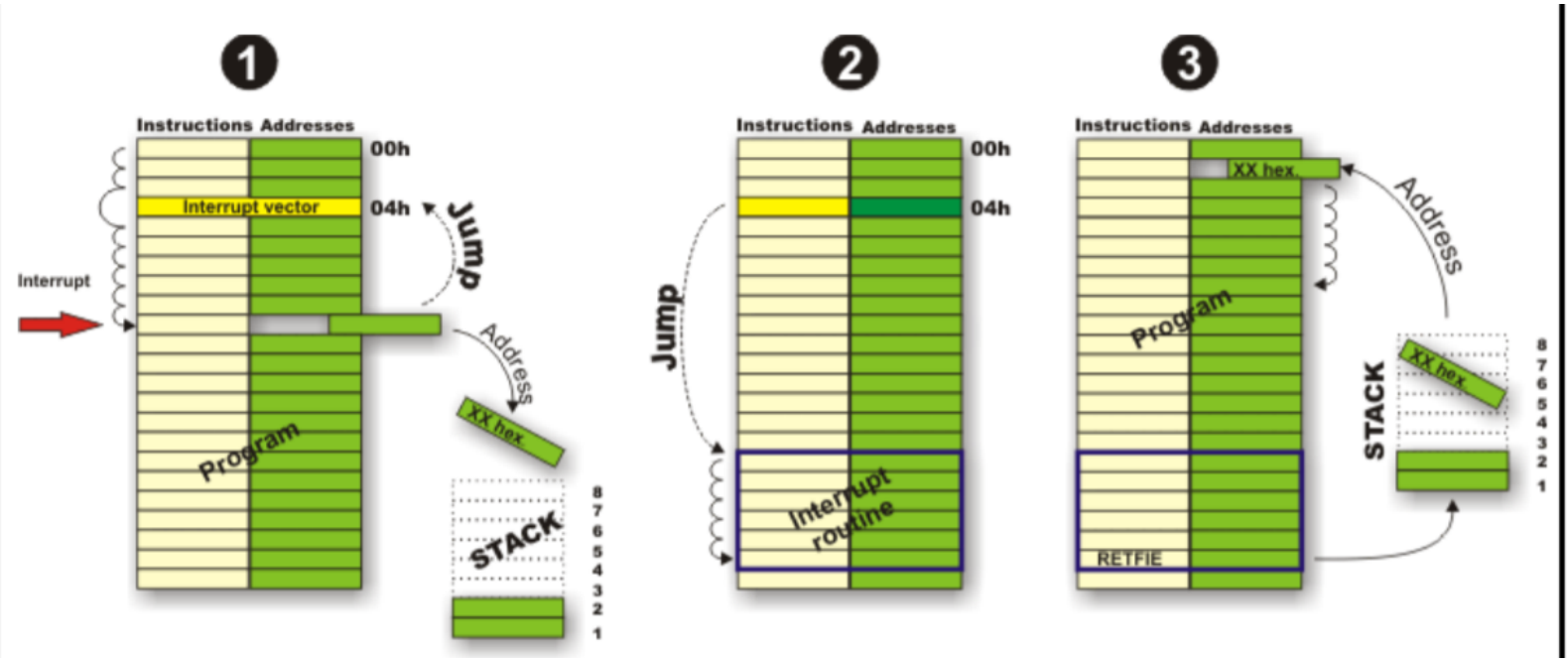
Po zakończeniu procedury obsługi przerwania

› Adres ze stosu jest umieszczany w liczniku programu PC
› Ustawiany jest globalny bit przerwań(i=1 odblokowanie)
› To powoduje kontynuację uprzednio przerwanego programu
› Przynajmniej jedna instrukcja zostanie wykonana
zanim nastąpi inne przerwanie

Jeśli wykorzystujemy np. wektor INT0 to pod adresem $002 w pamięci musimy zadbać o skok do podprogramu, który ma być wykonany w trakcie przerwania:

.org 0x0002
jmp PODPROGRAM

https://www.youtube.com/watch?v=qG1S6LIB050&list=PLqM6t4KYULi7Am36PsmjkE_qYmVGa8P_m&index=10 od początku



Wektor, nr	Adres pamięci programu	Źródło	Opis przerwania
1	$000	RESET	Pin zewnętrzny, Power-on reset, Brown-out reset, Watchdog reset, JTAG reset
2	$002	INT0	Zewnętrzne przerwanie INT0
3	$004	INT1	Zewnętrzne przerwanie INT1
4	$006	TIMER2 COMP	Timer / Licznik 2 dopasowanie porównania
5	$008	TIMER2 OVF	Timer / Licznik 2 przepełnienie
6	$00A	TIMER1 CAPT	Timer / Licznik 1 zdarzenie przechwycenia
7	$00C	TIMER1 COMPA	Timer / Licznik 1 dopasowanie porównania A
8	$00E	TIMER1 COMPB	Timer / Licznik 1 dopasowanie porównania B
9	$010	TIMER1 OVF	Timer / Licznik 1 przepełnienie
10	$012	TIMER0 OVF	Timer /Licznik 0 przepełnienie
11	$014	SPI, STC	Zakończona transmisja szeregowa
12	$016	USART, RXC	USART, Rx
13	$018	USART, UDRE	USART, pusty rejestr danych
14	$01A	USART, TXC	USART, Rx
15	$01C	ADC	Zakończona konwersja ADC
16	$01E	EE_RDY	EEPROM gotowy
17	$020	ANA_COMP	Komparator analogowy
18	$022	TWI	Interfejs szeregowy TWI (I2C)
19	$024	INT2	Zewnętrzne przerwanie INT2
20	$026	TIMER0 COMP	Timer / Licznik 0 dopasowanie porównania
21	$028	SPM_RDY	Store Program Memory Ready

3. **PC (Program Counter) -licznik rozkazu/programu W PC jest adres i trzeba wyjaśnić czego to jest adres i jak to działa.**

-wartość licznika rozkazów to adres komórki pamięci programu z następnym rozkazem do wykonania.

Podczas przerwań wartość licznika programu jest przechowywana w pamięci stosu.

Wartość PC pokazuje zawsze ta żółta strzałka w AVR.

PC resetuje się przy wyłączeniu urządzenia

PC ma 16 bitów- 8 w rejestrze o nazwie PCH i 8 w PCL (high,low).

PC adresuje pamięć programu i robi to automatycznie, ale my pisząc program możemy wpływać na to że zawartość PC się zmienia.

W normalnym przypadku po wykonaniu instrukcji wartość tego rejestru jest zwiększana o 1. W przypadku instrukcji warunkowych lub skoku, wartość jest zwiększana o 2 lub 3 (w zależności od instrukcji) albo zmieniana na adres instrukcji do której skacze procesor.

•Jak piszemy CALLa w kodzie i każemy skakać procesorowi do podprogramu to to jest nasza decyzja że wpłyniemy na zmianę wartości PC.

CALL powoduje zapisanie obecnej wartości rejestru PC na stosie a następnie skok (zmianę wartości PC) do pierwszej instrukcji znajdującej się za etykietą podprogramu. Po napotkaniu RET'a wartość PC jest odczytywana ze stosu, inkrementowana o 1 i ustawiana w rejestrze PC.

•PC adresuje pamięć programu i robi to automatycznie, ale my pisząc program możemy wpływać na to że zawartość PC się zmienia
Na etapie asemblacji każda instrukcja trafia w odpowiednie miejsce w pamięci. Domyślnie dzieje się to w sposób sekwencyjny. Jedna instrukcja pod drugą. Możemy jednak wpłynąć na miejsce docelowe instrukcji/miejsce od którego mają zostać zapisywane instrukcje za pomocą dyrektywy .org np. .org 0x0100

Mnemonics	Operands	Description	Operation	Flags	Clks
RCALL	k	Relative Subroutine Call	PC ← PC+k+1	None	3
ICALL		Indirect Call to (Z)	PC ← Z	None	3
CALL	k	Direct Subroutine Call	PC ← k	None	4
RET		Subroutine Return	PC ← STACK	None	4
RETI		Interrupt Return	PC ← STACK	I	4

RCALL przesunięcie k będzie liczone względem tego PC+1. Skacze pod adres w obrębie +/- 2 kilosłów. Do komórki wskazywanej przez wskaźnik stosu powędruje PC+1 (adres pierwszego rozkazu po RCALLU). Jeden bajt z dwubajtowego adresu (PC+1) idzie pod wskaźnik SP, drugi poszedł pod wskaźnik SP-1, a kolejna dekrementacja ustawiła adres wierzchołka stosu na SP-2. I to jest msc do następnego zapisu. I wtedy do PC wędruje info gdzie trzeba skoczyć, czyli PC+1 i względem tego przesunięcie w kodzie U2 (adr2k).

CALL adres pod który trzeba skoczyć jest przygotowany wcześniej w rejestrze Z. Do podprogramu sięgamy CALLami a z podprogramu wracamy RETami. RETI zmienia flagę I. Jak mamy pełen skok to na stos wędruje PC+2. CALL w programie zajmuje komórkę o adresie PC i komórkę o adresie PC+1, więc następny rozkaz do wykonania jest pod PC+2.

4. **Mapa pamięci -** na samym początku trzeba powiedzieć, że w każdej komórce znajduje się 16-bitowy ciąg zer i jedynek będący kodem rozkazu.

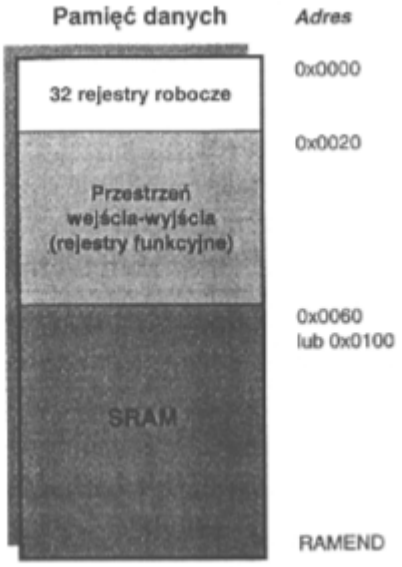
Typy pamięci(3):

1-FLASH- pamięć programu (kod programu, tablice stałych) - nieulotna pamięć. Pozwala na zapisywanie i kasowanie plików w dowolnym czasie.

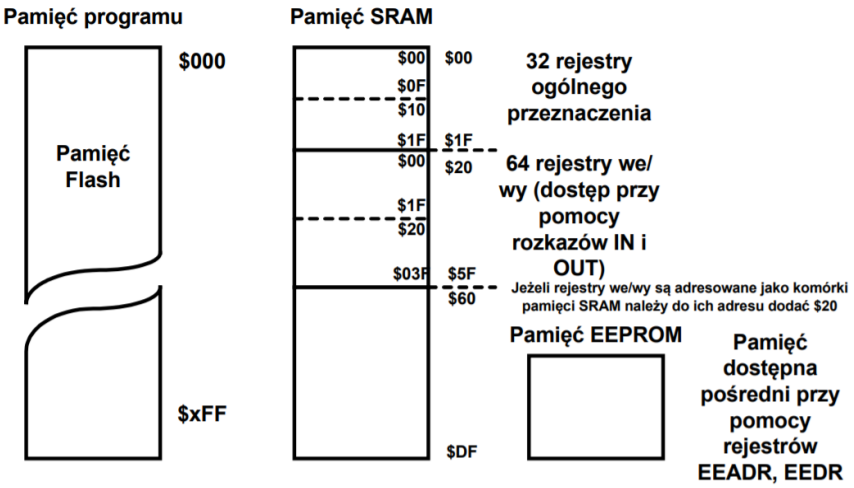
2-PAMIĘĆ DANYCH TYPU SRAM 1kB- pamięć danych -Pamięć buforująca między pamięcią operacyjną i procesorem. Jest to pamięć **ulotna** przy wyłączeniu zasilania.
-przechowuje zmienne

-umieszczony tu jest także **stos** sprzętowy(właściwie w INTERNAL SRAM) (obsługa przerwań i wywołań funkcji, odłożenie na niego bieżącej wartości licznika rozkazów i po zakończeniu działania zdejmowanie go)
-umieszczone są tu **rejestry** robocze r0-r31
PS.Sześć ostatnich rejestrów roboczych (R26-R32) może być używane jako trzy 16-bitowe rejestry wskaźnikowe (X, Y, Z)
-są tu umieszczone REJESTRY SPECJALNE, NP. rejestry I/O (w rejestrach IO jest SP), REJESTR SREG(STATUS REGISTER),

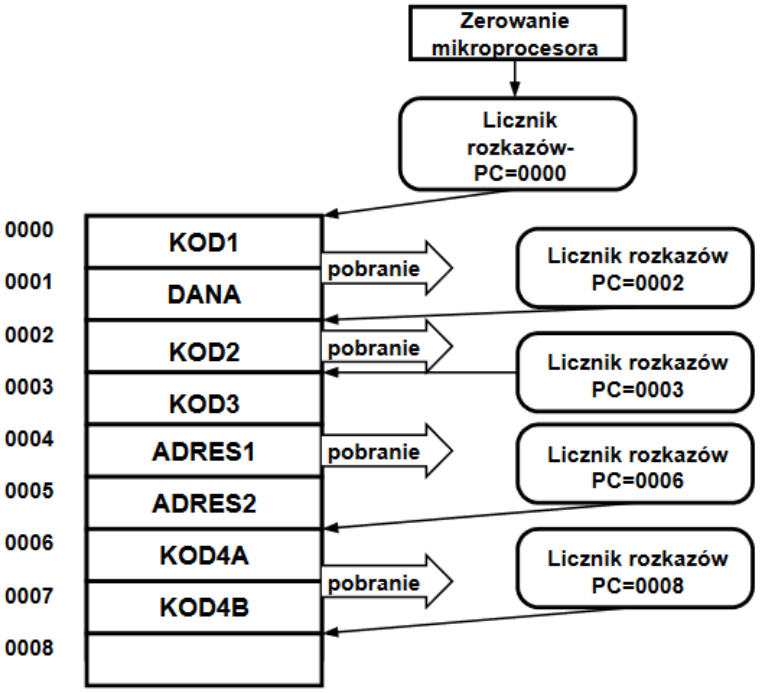
3-pamięć EEPROM (przechowuje zmienne lub tablice stałych, które po wyłączeniu zasilania nie ulegają skasowaniu).Pamięć dostępna pośrednio przy pomocy rejestrów EEADR, EEDR



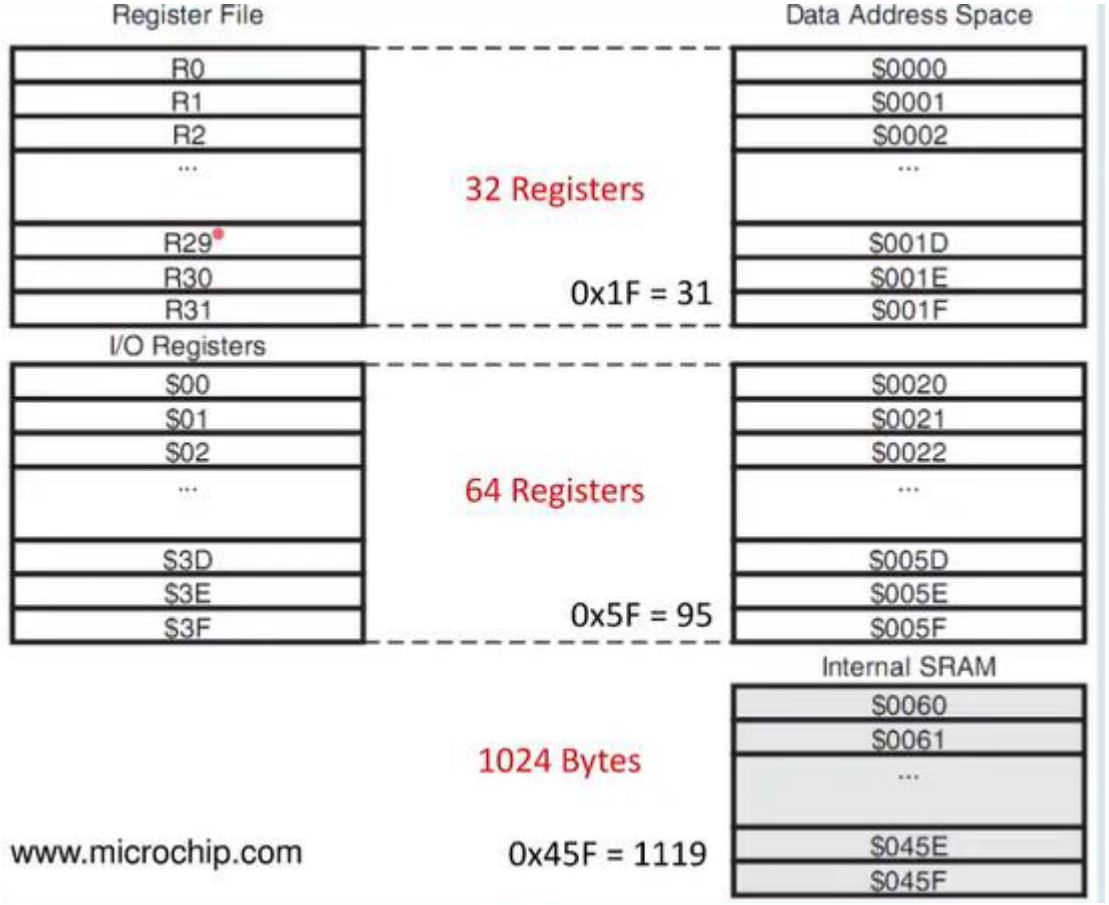
Przestrzeń adresowa pamięci danych



Wykonywanie programu przez procesor



-



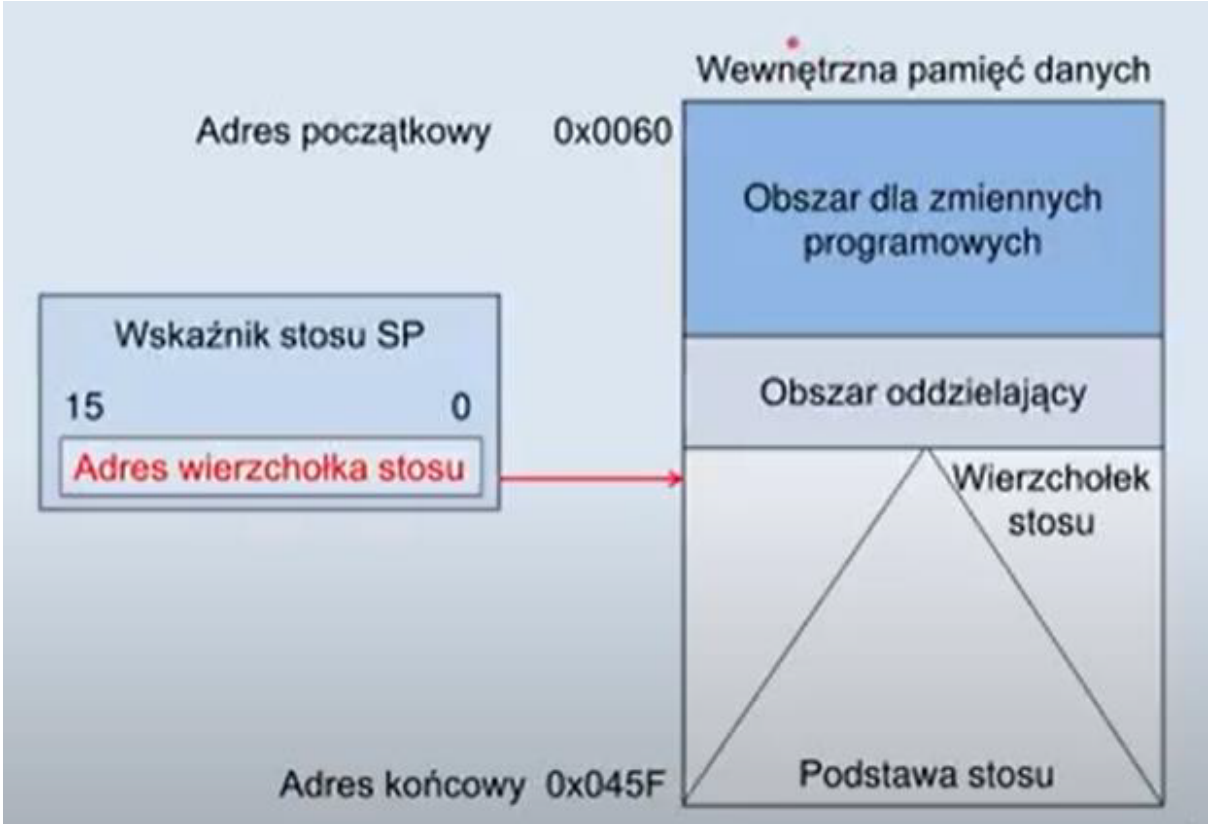
Rejestry robocze - Jednostka **ALU** umożliwia wykonywanie operacji arytmetycznych i logicznych między rejestrami, między stałą a rejestrem i operacji na pojedynczym rejestrze. Po wykonaniu operacji uaktualniany jest rejestr statusowy SREG dający info o rezultacie. Sześć ostatnich rejestrów pełni funkcje trzech 16-bitowych rejestrów wskaźnikowych (X, Y, Z). Daje to możliwość wykonywania szybkich przeliczeń danych. Z służy jeszcze przy rozkazach LPM (Load Program Memory).
Jak włączymy komputer to wszędzie dla każdego rejestru stan początkowy ustawi się jako zera. Ale taki stan początkowy ustawia się tylko w środowisku, które udaje procesor. W prawdziwym procesorze tak nie jest. Te rejestry X Y Z w prawdziwym procesorze w technologii CMOS ustawiają się na same jedynki.

ze względu na działanie niektórych instrukcji rejestry robocze podzielone są na r0-r15 i r16-r31.
Od r16 do r31 mamy bezpośredni dostęp poprzez control lines (magistrale).
Dane ładujemy do rejestrów od r16 do r31. Ale jak masz coś dalej z tymi danymi robimy, to przerzucamy je do niższych rejestrów, bo do tych od 16 do 31 zaraz znowu będziemy coś ładować.

out (prześlij zawartość wskazanego rejestru roboczego do wskazanego rejestru I/O).
Input (prześlij zawartość ze wskazanego I/O do wskazanego rejestru roboczego).

Rejestry I/O - służy do przechowywania rejestrów funkcyjnych (specjalnych), umożliwiających sterowanie pracą procesora i wewnętrznych układów peryferyjnych. Moduł przerwań ma swoje rejestry kontrolne w przestrzeni I/O z dodatkowym globalnym bitem odblokowującym przerwania w rejestrze statusowym SREG.
Umieszczone w pamięci danych SRAM.

Internal SRAM - służy do przechowywania zmiennych programowych. Dostęp do pamięci SRAM trwa 2 cykle zegarowe, nie można przeprowadzać na niej operacji arytmetyczno-logicznych w sposób bezpośredni. Znajduje się w niej **stos**.
Wydzielony obszar pamięci służący do przechowywania danych, adresów powrotów z procedur, adresów powrotów z przerwań. Działa jak kolejka typu FI-LO (First Input-Last Output).Adres wierzchołka stosu pokazuje wskaźnik stosu. Każdy program musi zainicjalizować wskaźnik stosu SP, zanim zostanie wywołane przerwanie czy podprogram. **Przy zapisie na stos, SP jest zmniejszany..**



5. **Rejestr SREG -** Rejestr statusowy SREG zawiera informacje o rezultacie ostatnio wykonywanej operacji arytmetycznej. Informacja ta może być użyta przez instrukcje warunkowe sprawdzające stan wyniku poprzedniej operacji.
   Rejestr statusowy jest aktualizowany po każdej operacji wykonanej przez ALU. Nie zawsze musi być zmieniana cała zawartość SREG. Zależnie od instrukcji, może to być np. tylko jeden bit. Dzięki wykorzystaniu tych właściwości możliwe jest tworzenie bardziej zoptymalizowanego kodu pod względem szybkości i objętości.
   Zawartość rejestru SREG nie jest automatycznie zapamiętywana przy wejściu w kod obsługi przerwania i odzyskiwana po jego wykonaniu. Należy to wykonywać programowo.

   Rejestr ten znajduje się **w pamięci danych, w rejestrach specjalnych.**

   Stan początkowy dla SREG to same zera.

7	6	5	4	3	2	1	0
I	T	H	S	V	N	Z	C
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
0	0	0	0	0	0	0	0

   Na najmłodszych bitach tego rejestru (0 i 1) mamy coś, co jest używane nagminnie. Jeżeli używamy instrukcji arytmetycznych (dodawania np) no to wiadomo, że jak się

dodaje dwie liczby 8-bitowe no to wynik albo się zmieści na 8 bitach, lub nie. Może powstać liczba 9 bitowa którą musimy tak naprawdę zapisać na 16 bitach.
Przy powstawaniu tej liczby 9-bitowej – dziewiąty bit to jest przeniesienie i ta operacja ustawia **bit 0 (C – Carry)** na jedynkę.
Jeśli w wyniku takiej operacji dodalibyśmy dwie liczby w taki sposób, że powstanie przeniesienie, ale jednocześnie 8 młodszych bitów się wyzeruje, to wtedy te zera wpisują się w rejestr wynikowy – on rzeczywiście przyjmuje wartość samych zer. Carry się ustawia, ale ponieważ rejestr wynikowy się wyzerował, to ustawi się też **bit 1 (Z – Zero)**. Ten bit zawsze oznacza – przy różnych operacjach – że wynik ostatniej operacji arytmetyczno-logicznej przyniósł 0 dziesiętnie, czyli 8 zer binarnie. Gdybyśmy dekrementowali r16 (który miał zapisaną wartość 10), no to po pierwszej dekrementacji będzie miał 9, potem 8,..., wreszcie będzie miał 0. I jak się wyzeruje, to wskaźnik Z ustawiony do tej pory na 0 ustawi się na 1 i będzie informował że wynik ostatniej operacji przyniósł 8 bitowe zero.

Bit 2 czyli N – Negative. Jego flaga ustawi się na „1” kiedy wynik ostatniej operacji arytmetycznych w kodzie U2 będzie ujemny.

Bit 3 czyli V – Overflow. Jeżeli dojdzie do przekroczenia zakresu w kodzie U2

Bit 4 czyli S – Sign. Niezależnie od wyniku operacji arytmetycznych to tutaj się ustawi bit znaku widziany jako bit w kodzie U2. S=1 gdy wynik jest jawnie ujemny i nie nastąpiło przepełnienie

Bit 5 czyli H – Half Carry. Jak mamy 8 bitowe słowo to można je podzielić na pół (na dwie tetrady) i jak dodajemy dwa takie słowa to jeżeli: w tym samym miejscu wystąpi „1” to będzie przeniesienie.I właśnie wtedy się ustawia Half Carry. Używany jest powszechnie przy wszystkich translacjach np z systemu dziesiętnego na dwójkowy.

Bit 6 czyli T – Bit copy storage. -rejestr bitowy do kopiowania bitów-Przechowanie kopiowanego bitu. Instrukcje kopiowania bitu BLD i BST używają T jako bit źródłowy lub docelowy do operacji bitowej.

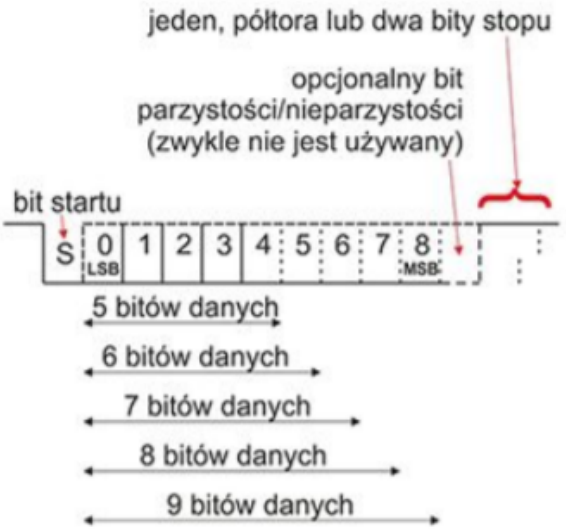
Bit 7 czyli I – Global Interrupt enable. Bit informujący o możliwości odbycia przerwania. Do odblokowania służy instrukcja Set Interrupt SEI(I=1), do zablokowania Clear Interrupt CLI (I=0). Znacznik I jest automatycznie zerowany po wejściu do procedury obsługi przerwania i ustawiany po wyjściu z niej wskutek wykonania rozkazu RETI

6. **Kiedy odbiornik zaczyna przyjmowanie (uart)**

Po przejściu ze stanu jałowego (podstawowo 1) kiedy otrzyma **bit startu** (czyli stan niski 0). Każdy bit trwa 16 impulsów.

Przesyłane interfejsem UART/RS2232 informacje formowane są z ramek składających się z:

- bitu startu,
- 5, 6, 7 lub 8 bitów danych,
- bitu parzystości (opcjonalnego, zwykle pomijanego),
- 1, 1,5 lub 2 bitów stopu (rys. 2).



Rys. 2. Ramka protokołu UART/RS232

7. **Mechanizm nadawania i odbioru UART**

bylo to w sprawozdaniu 2

Nadawanie: Bity nadawane są z UDR. Trzeba sprawdzić **UDRE** z UCSRA:

•Bit 5 **UDRE-** DATA REGISTER EMPTY: informuje o tym, czy rejestr danych UDR jest pusty i możemy wpisać nową daną do wysłania - dzięki temu nie nadpiszemy danych podczas wysyłania UDRE==1 → BUFOR JEST PUSTY

-i skąd będziesz wiedzieć kiedy próbka została dostarczona??RXComplete

***Odbiór*:** UDR jako bufor, sprawdzamy **RXC** z UCSRA czy jest coś do odebrania i należy pamiętać o odblokowaniu **RXEN** i **TXEN** z UCSRB i wybraniu parametrów ramki z UCSRC

UCSRA:

•Bit 7 **RXC**-RECEIVE COMPLETE: Zakończono odbiór znaku RCX==0 →BUFOR ODBIORU JEST PUSTY

UCSRB:

- •Bit 4 RXEN- RECEIVER ENABLE
- •Bit 3 TXEN- TRANSMITER ENABLE

USART Control and Status Register A-

Bit	7	6	5	4	3	2	1	0	
	RXC	TXC	UDRE	FE	DOR	PE	U2X	MPCM	UCSRA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

•Bit 7 RXC-RECEIVE COMPLETE:
Zakończono odbiór znaku
RCX==0 →BUFOR ODBIORU JEST PUSTY

•Bit 6 TXC-TRANSMIT COMPLETE:
Zakończono transmisję całej ramki
TXC==1→BUFOR TRANSMISJI JEST PUSTY

•Bit 5 UDRE- DATA REGISTER EMPTY:
informuje o tym, czy rejestr danych UDR jest pusty i możemy wpisać nową daną do wysłania (jest to sprzętowe rozwiązanie portu szeregowego)- dzięki temu nie nadpiszemy danych podczas wysyłania
UDRE==1 → BUFOR JEST PUSTY

•Bit 4 FE- FRAME ERROR:
Kontrola błędów ramki
FE==1 →kolejny znak w buforze odbioru został odebrany z błędem

•Bit 3 DOR-DATA OVER RUN
DOR==1 →przepełnienie ramki

•Bit 2 PE-PARITY ERROR:
Kontrola Parzystości

•Bit 1 U2X- DOUBLE THE USART TRANSMISSION SPEED
Dwukrotne zwiększenie prędkości transmisji

•Bit 0 MPCM- MULTI-PROCESSOR COMMUNICATION MODE
Wieloprocesorowa komunikacja

USART Control and Status Register B-

Bit	7	6	5	4	3	2	1	0	
	RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8	UCSRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

•Bit 7 RXCIE- RX COMPLETE INTERRUPT ENABLE
•Bit 6 TXCIE- TX COMPLETE INTERRUPT ENABLE
•Bit 5 UDRIE- DATA REGISTER EMPTY INTERRUPT ENABLE
powyższe bity dotyczą przerwań

•Bit 4 RXEN- RECEIVER ENABLE
•Bit 3 TXEN- TRANSMITER ENABLE
on/off odbiornika, domyślnie wyłączone
on/off nadajnika, domyślnie wyłączone

Linie nadajnika i odbiornika są na porcie PB0 i PB1- normalnie te linie funkcjonują jako zwykłe linie wejściowo-wyjściowe. Jeśli chcemy uruchomić nadajnik musimy aktywować powyższe bity (ustawić 1).

•Bit 2 UCSZ2- CHARAKTER SIZE
Razem z pozostałymi bitami UCSZ1 i UCSZ0 z rejestru UCSRC będzie odpowiadał za liczbę bitów
•Bit 1 RXB8- RECEIVER CATA BIT8
•Bit 0 TXB8-TRANSMITER DATA BIT 8
Powyższe 2 bity wykorzystywane są przy transmisji 9 bitowej w ramce

USART Control and Status Register C-

Bit	7	6	5	4	3	2	1	0	
	URSEL	UMSEL	UPM1	UPM0	USBS	UCSZ1	UCSZ0	UCPOL	UCSRC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	0	0	0	0	1	1	0	

•Bit 7 URSEL - REGISTER SELECT
Ten bit związany jest, że pod tym samym adresem umieszczono dwa rejestry-UCSRC i UBRRH. Zastosowano takie rozwiązanie prawdopodobnie z braku przestrzeni adresowej. Wybór, do którego rejestru chcemy się odwołać odbywa się właśnie przez ten bit nr 7

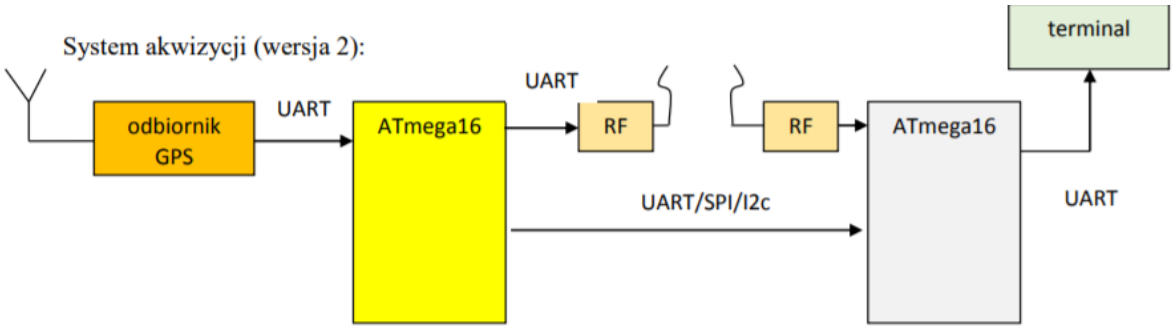
•Bit 6 UMSEL - MODE SELECT
Ustala tryb transmisji. Domyślnie UMSEL==0 →transmisja asynchroniczna

•Bit 5:4 UPM1:0 PARITY MODE
Bity kontroli parzystości. Domyślnie UPM1:0==0 → wyłączona kontrola parzystości

•Bit 3 USBS - STOP BIT SELECT
Ustala liczbę bitów stopu. Domyślnie USBS==0 → 1 BIT STOPU

•Bit 2 :1 UCSZ1:0 CHARAKTER SIZE
Razem z UCSZ2 w rejestrze UCSRB. Możemy wysyłać od 5 do 9 bitów w ramce.
Domyślne dla 8 bitów: UCSZ2==0 UCSZ1==1 UCSZ0==1

•Bit 0 UCPOL CLOCK POLARITY
Bit związany z transmisją synchroniczną(polaryzacją zegara).



Zadania realizowane przez system:

8. **Jak to jest że wychodzące elementy czasowe są równiutkie, jak ten czas jest odmierzany**
   Impulsy są nadawane co określony odstęp czasu, jest on uzależniony od oscylatora znajdującego się w procesorze z uwzględnieniem preskalera który jest ustawiany programowo.

9. Schemat bloku akwizycji w 3 wersjach

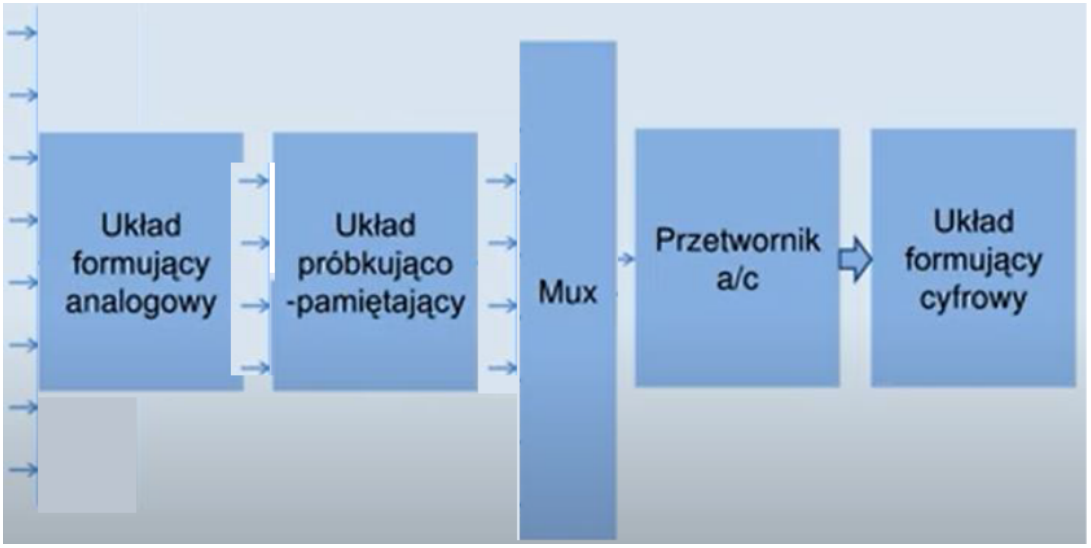
1:

układ który przetwarza tylko 1 kanał (sytuacja z obrazka)



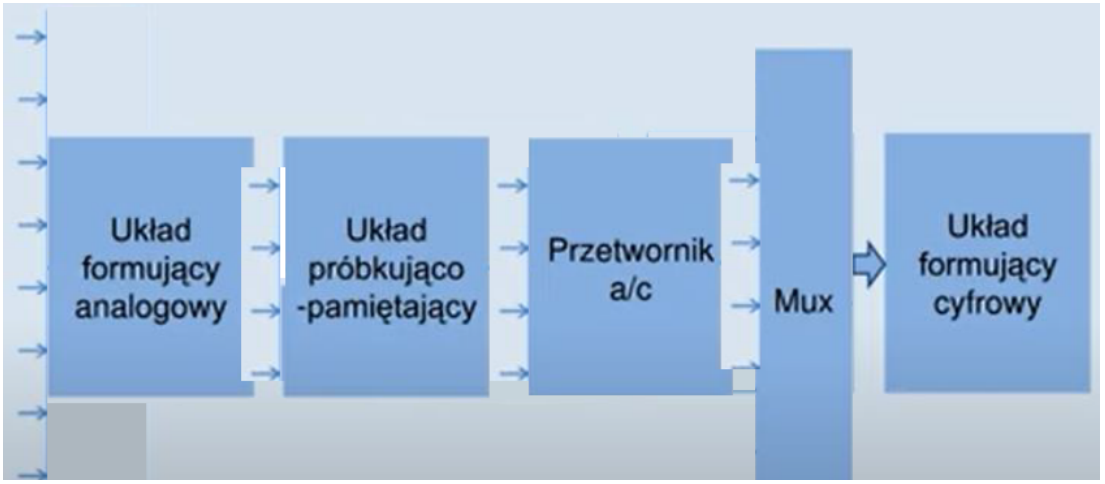
2: To Układ FA -> UKŁAD PP - > MUX -> AC -> FC - mieszane zbieranie próbek

jednokanałowy przetwornik AC ale wielokanałowy układ próbkująco-pamiętający



3: To układ FA -> UKŁAD PP -> AC -> MUX -> FC - jednoczesne zbieranie próbek ? (nie jestem pewien nazwy)

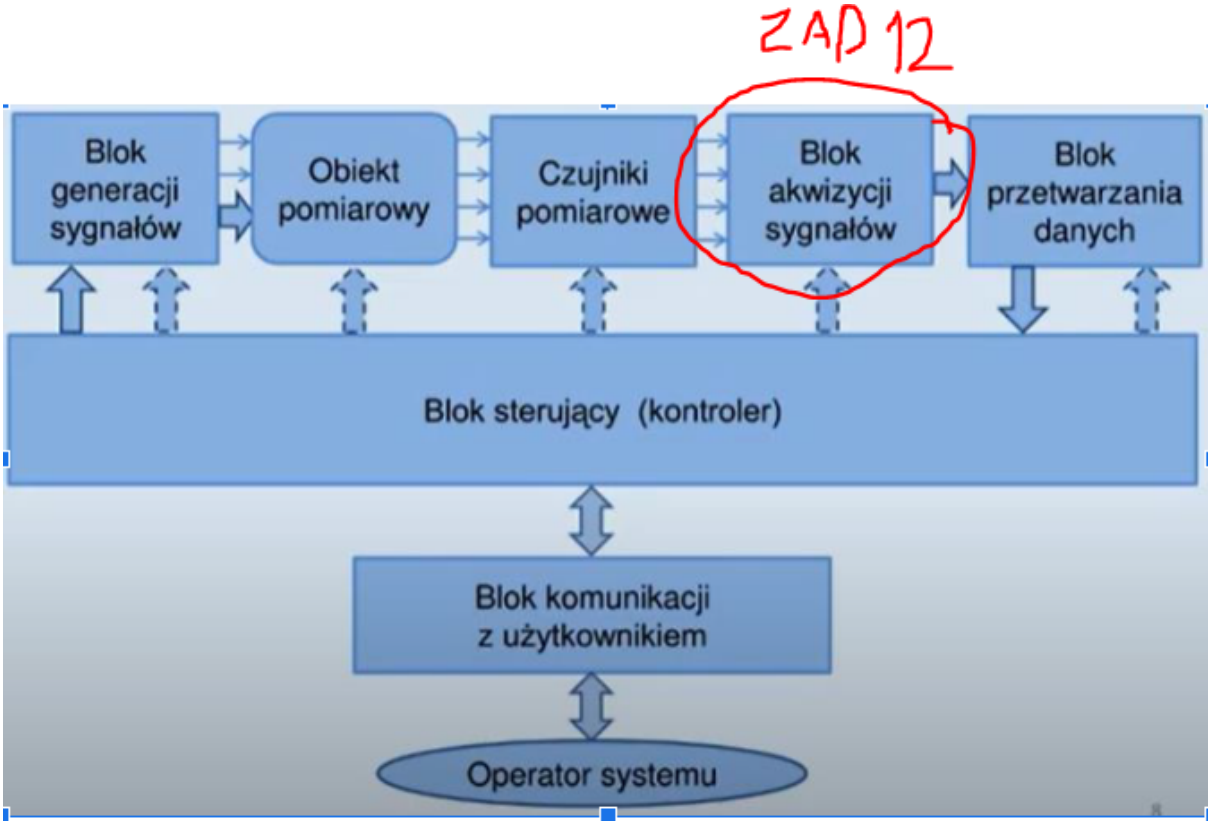
cudo co potrafi kilka na raz i potem tylko mówi procesorowi pod jakim adresem muxa ma jaki przetworzony sygnał (sytuacja 3cia)



Mux - To jest zbiór kluczy elektronicznych, które możemy postrzegać jako takie twory, że sygnał sobie wchodzi z lewej strony a te klucze mają sygnał sterujący i klucz albo zamyka albo otwiera. Więc można powiedzieć, że ten mux ma wbudowaną część cyfrową, która przy tej liczbie strzałek którą widzimy miałaby 3 wejścia: Wejścia sterujące (C B A). Każda kombinacja zer i jedynek byłaby adresem podawanym z np mikroprocesora i każda taka informacja adresowa wybiera jeden klucz. Po lewej mamy indywidualne wejścia. Podanie kodu adresowego wybiera klucz i powoduje jego zamknięcie. Nigdy nie powinna zaistnieć sytuacja, że dwa klucze zostaną jednocześnie zwarte. Bo wtedy na wyjście muxa (które jest jedną strzałką) poszłyby dane z dwóch kluczy i wystąpiłby konflikt.
Jeżeli zamkniemy 1 klucz i nie interesuje nas co dzieje się na innych obiektach pomiaru, co przynoszą sygnały z pozostałych czujników, to mówimy o tzw pracy jednokanałowej.

Układ FA - jak wolnozmienny czy szybkozmienny jest przebieg wejściowy. Wbudowywany jest tłumik. Dzięki niemu mamy możliwość zmniejszenia poziomu sygnału. Wiemy jaki jest współczynnik tłumienia. Powinniśmy ustawić tam np podział napięcia przez 10.
Układ konstruowany jest tak, żeby na wejściu miał wtórnik napięciowy na wzmacniaczu operacyjnym – wzmocnienie = 1 i bardzo duża rezystancja wejściowa R2

10. Narysować schemat systemu pomiarowego – np. schemat z nie wypełnionymi bloczkami niektórymi i trzeba je nazwać



Blok generacji sygnałów wykorzystywany jest do wytwarzania sygnałów wymuszających (programowane źródła napięć i prądów), do generacji sygnałów wzorcowych (odniesienia) oraz do wytwarzania sygnałów sterujących elementami obiektu pomiarowego. Blok ten wymaga jednego lub kilku przetworników C/A w celu wytworzenia sygnałów analogowych.

Obiekt pomiarowy - oddziałujemy na obiekt pomiaru w sposób analogowy lub cyfrowy. Z obiektu pomiaru ściągamy sygnały analogowe (no chyba że testujemy coś cyfrowego) i niesiemy to na czujniki pomiarowe.

Czujniki pomiarowe to elementy umożliwiające odbiór informacji z obiektu fizycznego, którego parametry podlegają identyfikacji w procesie pomiarowym; czujniki wielkości fizycznych, czujniki wielkości chemicznych. Konstrukcyjnie czujnik stanowi przetwornik złożony na ogół z kilku prostszych przetworników mechanicznych i elektrycznych umieszczonych we wspólnej obudowie umożliwiającej zamontowanie go na obiekcie badanym lub w jego bezpośrednim otoczeniu.

Blok akwizycji należy do najważniejszych bloków funkcjonalnych systemu pomiarowego.
Zamienia sygnał A na C
W analogowych systemach pomiarowych, na etapie akwizycji sygnału pomiarowego, muszą być zastosowane elementy do kondycjonowania sygnału.
W ogólności kondycjonowanie to wstępna normalizacja sygnału analogowego, a więc **dostosowanie postaci i zakresu zmienności sygnału do formy wymaganej na wejściu przetwornika A/C.** Do zadań układów kondycjonowania należą:
- wytworzenie odpowiedniego poziomu sygnału wyjściowego dopasowanego do zakresu wejściowego przetwornika A/C,
- ograniczenie pasma częstotliwościowego sygnału,
- usunięcie zakłóceń,
- linearyzacja,
- przełączanie sygnałów analogowych z czujników pomiarowych.

Porównanie sygnału pomiarowego z wzorcem. W tym bloku dokonuje się po pierwsze porównanie z wzorcem a po drugie zamiana sygnału analogowego na sygnał cyfrowy. Żeby ten sygnał poszedł dalej jako strumień danych w transmisję. I żeby odległość nie stała się przyczyną niewiarygodności danych.
Jeżeli chodzi o przetwarzanie analogowo-cyfrowe (AC) to w jego skład wchodzą 3 operacje:

Próbkowanie
Kwantowanie
Kodowanie

Blok przetwarzania danych jest odpowiedzialny za cyfrową obróbkę sygnałów pomiarowych zgodnie z przyjętym algorytmem. Umieszczanie próbek w adresowalnych komórkach pamięci FIFO i odtwarzanie jako pierwszej próbki do transmisji tej, która jako pierwsza została przesłana.

blok sterowania (kontroler) – zapewnia prawidłową pracę systemu, musi zawierać pamięć programu oraz układ realizujący ten program. Koordynuje działanie wszystkich bloków funkcjonalnych, organizuje przepływ wszelkich informacji i odpowiada za realizację algorytmu działania systemu. Do podstawowych funkcji kontrolera należą: sterowanie aparaturą pomiarową i obiektem, zbieranie i magazynowanie danych pomiarowych, przetwarzanie danych pomiarowych;

Blok komunikacji z użytkownikiem– umożliwia użytkownikowi odbieranie i wprowadzanie informacji do systemu;

11. Dopasowanie procesora do systemu

Architektura CISC– ang. Complex Instruction Set Computers

-występowanie złożonych, specjalistycznych rozkazów (instrukcji), które wymagają od kilku do kilkunastu cykli maszynowych (zmienna liczba cykli),
-szeroka gama trybów adresowania (skomplikowana konstrukcja dekoderów adresu),
-stosunkowo długa lista rozkazów procesora.

Wady architektury CISC:

- zbyt długa lista rozkazów
- część z nich jest rzadko używana,
-zbyt dużo czasu traci się na operacje przepisania z pamięci do rejestrów i odwrotnie,
-ogólnie mała efektywność w obliczeniach numerycznych.

Architektura RISC– ang. Reduced Instruction Set Computers

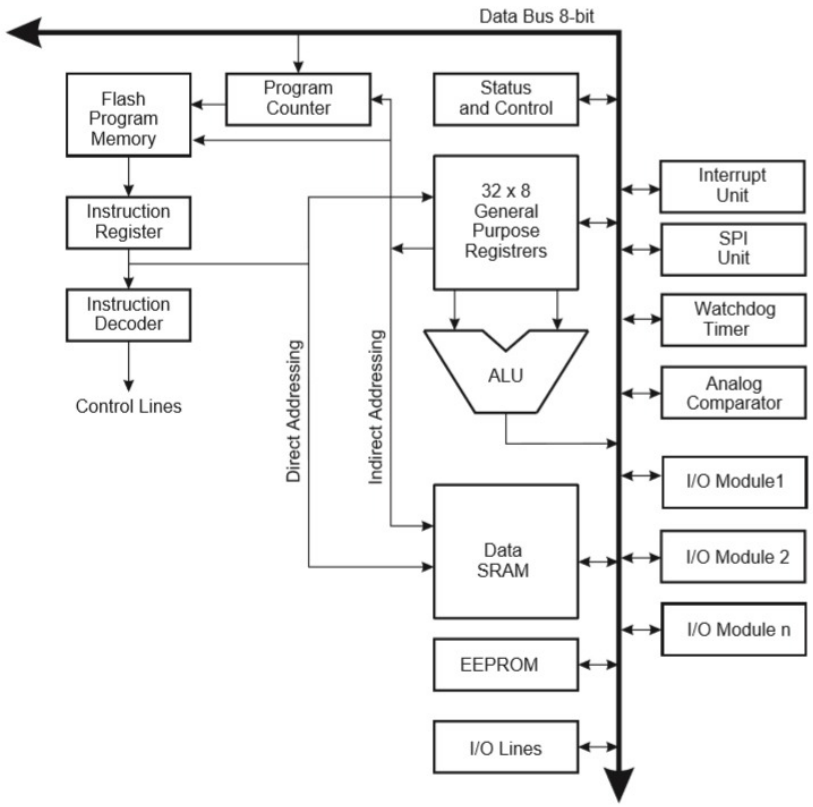
-zredukowana liczba rozkazów do niezbędnego minimum,
-redukcja trybów adresowania, dzięki czemu kody rozkazów są prostsze,bardziej zunifikowane, (upraszcza dekoder rozkazów),
-ograniczenie komunikacji pomiędzy pamięcią, a procesorem,
-przetwarzanie potokowe
- równoległe wykonywanie rozkazów.

Obecnie popularne procesory z punktu widzenia programisty są widziane jako CISC, ale ich rdzeń jest RISC-owy. Rozkazy CISC są rozbijane na mikrorozkazy, które są następnie wykonywane przez RISC-owy blok wykonawczy.

Architektury:

–**Von Neumanna -** Wspólna przestrzeń adresowa dla pamięci kodu i danych. Wspólne magistrale:-danych,-adresowa,-sterująca.
–**Harvardzka -** Osobne magistrale adresowe i danych dla programu i danych.
–**Harvardzka zmodyfikowana -** Wspólna magistrala danych i adresowa, ale pamięci rozdzielone dzięki osobnym sygnałom sterującym.

12. Diagram blokowy



13. Pytania z testów

.cseg - wybór pamięci programu do umieszczenia kodu programu
ldi - pozwala zapisać liczbę z zakresu 0-255 do jednego z rejestrów r16-r31
cpse - porównuje 2 rejestry, pomija kolejną instrukcję, gdy warunek jest spełniony
DDRx - ustalenie kierunku pracy linii I/O
PORTx - wysyłanie danych na linie portu
PINx - odczyt stanu linii I/O
Każda linia I/O jako wejście lub wyjście.
sbis - testuje stan wybranej linii portu I/O
cpi - porównuje zawartość rejestru roboczego ze stałą
cbi - ustawia 0 na wybranej linii portu
Do powrotu z przerwania - RET i BSET(7)
Jeden 8-kanałowy przetwornik A/C
ADSC do rozpoczęcia ADC
A/C jest 10-bitowy

14. JMP

BRANCH INSTRUCTIONS

Mnemonics	Operands	Description	Operation	Flags	#Clocks
RJMP	k	Relative Jump	PC ← PC + k + 1	None	2
IJMP		Indirect Jump to (Z)	PC ← Z	None	2
JMP	k	Direct Jump	PC ← k	None	3

JMP - Ma swój argument i kod operacji. Argument jest 16 bitowym adresem. Jest to skok bezpośredni. Zmienia kolejność wykonywania rozkazów i jest zaznaczany jako „do PC wędruje wartość która jest adresem i PC przyjmuje wartość NIE PC + 2 tylko przyjmuje wartość k" czyli następny rozkaz wykonywany nie będzie następnym po jmp tylko będzie rozkazem, który znajduje się pod jakimś innym adresem (K).

RJMP - ten rozkaz jest wykonywany szybciej. Jest używany bardzo często. Jest oszczędny. Ale będzie miał krótszy zasięg. Dlaczego? Bo w argumencie k nie jest zapisany adres bezpośredni, tylko jest zapisana liczba w kodzie U2 – u nas podawana przez etykietę. Ta liczba nie jest zapisywana na 16 bitach, ona operuje na obszarze +/- 2KB. Więc on nie obejmuje całej pamięci. Jakbyśmy chcieli, żeby nas przeniósł do dalekich adresów, to on tego nie zrobi. Jak procesor wykonuje ten rozkaz, to najpierw zwiększa PC +1, a potem nalicza przesunięcie k. On liczy przesunięcie względem pierwszego rozkazu po RJMP.

IJMP - argumentem do tego skoku (adresem dokąd skoczyć) jest zawartość rejestru Z. I mamy go tutaj jako pełne Z czyli pełne 16-bitów. Po pierwsze jest to skok o długim zasięgu, a po drugie my na ten adres możemy programowo wpływać.
Jeżeli my zapiszemy w programie jakąś etykietę, to JMP i RJMP stają się skokami sztywnymi. A w IJMP możemy zrobić coś takiego, że my ten rozkaz wpiszemy, ale możemy go wpisać w jednym miejscu,a w innych miejscach w programie zmieniać wartość tego rejestru Z. I wtedy kiedy potem skokiem wrócimy do tego rozkazu, to on będzie już operował na innym adresie. Mamy procesor i przy nim 1 przycisk.
A my chcemy się z procesorem połączyć i dawać mu różne polecenia. I wtedy można zrobić coś takiego, że wpisuje mu się program i wprowadzane jest coś takiego – zliczaj przyciśnięcia tego klawisza. Nastaw się na to, że będzie ich np 5.
Przyciśnięcie pierwsze oznacza reakcję 1, drugie oznacza reakcję 2 Itd Szóste przyciśnięcie będzie powrotem do reakcji 1 i tak w kółko. Wtedy procesor każde wciśnięcie klawisza zlicza i zmienia wartość Z na przypisaną temu numerowi. Następnie skacze do tego rozkazu, a ten rozkaz kieruje do fragmentu kodu, który odpowiada albo reakcji 1, albo 2, 3 itd. Przy pomocy jednego klawisza można w tym momencie obsłużyć 5 różnych podprogramów.

15. LD i ST

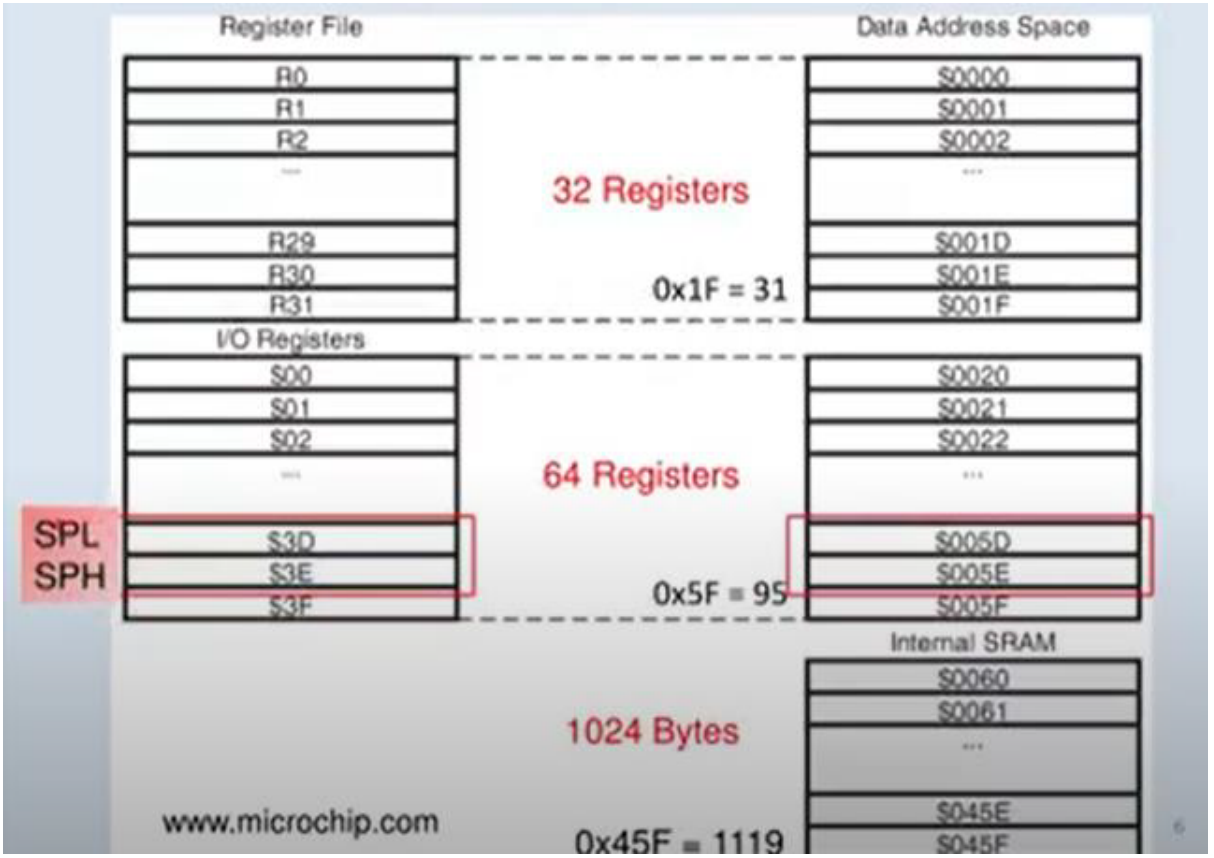
Rozkazy LD (load – załaduj) i ST(store – pamiętaj).
Zasady używania tych rozkazów są proste i wyglądają następująco:
Jeżeli mamy rozkaz LD to w domyśle jest „ładuj rejestr roboczy" czyli skądś trzeba pobrać daną i załadować ją do rejestru roboczego. Czyli jak widzimy instrukcję LD to od razu mamy wiedzieć, że będzie chodziło o rejestr roboczy.

Jak mamy rozkaz ST to mamy od razu wiedzieć, że będzie chodziło o zapamiętanie danej z rejestru roboczego (czyli przesłanie jej z rejestru dokądś).

Wykorzystamy rozkaz LD żeby przesłać nasze informacje z 0x300 do r0, a z r0 możemy te informacje przesłać do 0x100 za pomocą instrukcji ST.

16. **ADD, ADC**
    Najpierw używamy instrukcji clr. Ona w bardziej elegancki sposób zastępuje „ldi r2, 0”. To samo robimy z r1. Bo przygotowujemy się pod umieszczenie wyniku.
    Mamy dwa argumenty które będziemy dodawali (r16 i r17).
    R16 = 1111 0000
    R17 = 0000 1111
    Następnie prześlij zawartośd R16 do rejestru R0 po to, żeby argument w r16 zachować.
    Dodaj r0 do r17 to jest po to, żeby zachować argument z r17
    W wyniku tych operacji wynik będzie się pojawiał w innym rejestrze
    r0 = 1111 1111
    Ostatni rozkaz: adc r1, r2 niczego nie zmienia. Dodaje do r1 rejestr r2. Oba były wyzerowane. Ona oznacza, żeby dodać do r1 i r2 oraz Carry.
    Carry ustawiłoby się na 1 gdyby to dodawanie wyszło poza zakres i wystąpiłoby przeniesienie. Ale w tym przypadku tak się nie stało, więc mamy r1 = 0000 0000.
    To co tutaj zrobiliśmy to zarezerwowaliśmy na wynik dwa bity r1 oraz r0. R1 razem z R0 tworzą 16 bitowy wynik tej operacji. 0000 0000 1111 1111

17. **Wskaźnik stosu -** adres szczytu stosu na którym procesor może zapisywać dane lub je stamtąd odczytywać zgodnie z LIFO- czyli adres gdzie jest pusto i można zapisać



SPL i SPH to są młodsze i starsze bity wskaźnika stosu. We wskaźniku stosu znajduje się adres komórki pamięci będącej szczytem stosu.
Wskaźnik stosu inicjalizuje się wpisując do niego tę wartość: 0x045F (czyli adres końcowy wewnętrznej pamięci danych). Zawsze jak coś zapiszemy na stos, to wskaźnik stosu będzie miał adres komórki pamięci danych, do której należy wpisać kolejną daną. Wskaźnik stosu sam się inkrementuje i dekrementuje.

```
.org 0x00
.nolist
.include "m16def.inc"
.list

ldi r16, high (RAMEND)    ;ładuj MSB adresu
                          ;końca SRAM
out SPH, r16              ;inicjalizuj MSB
                          ;wskaźnika stosu
ldi r16, low (RAMEND)     ;ładuj LSB adresu
                          ;końca SRAM
out SPL, r16              ;inicjalizuj LSB
                          ;wskaźnika stosu
...
```

RAMEND jest symbolem zapisywanym binarnie na 16 bitach. Operator high wycina z 16 bitów starszy bajt, a my rozkazem out przepiszemy r16 na SPH (starszego bajtu wskaźnika stosu).

21. Co robi atmega po starcie

PC = 0

22. dodawanie 3 liczb

```
ldi r16, 255
ldi r17, 255
ldi r18, 255

add r0, r16
adc r1, r2

add r0, r17
adc r1, r2

add r0, r18
adc r1, r2
```

bity Carry mamy w r1



22. dodawanie 2 liczb 16bit

rozdzielam każdą z liczb na po dwa rejestry po 8 bit
dodaje te mlodsze części
dodaje starsze
uwzględniam przeniesienia jeśli jakieś wystąpiły-jak suma młodszych jest większa niż 255 to dodajesz do starszych
a jak suma starszych to daje Ci 3ci rejestr

23. program sumujący 1024 liczby z zewnętrznego źródła

tak jak zad 21
3 rejestry
i podsumowywuje
z uwzględnieniem carry zawsze

*24.*jak policzyc srednia arytmetyczna probek 8 bitowyhch-i nie chodzilo mu o dzielenie raczej o sume tego nie wiemy

1) Clear 2 consecutive RAM bytes before the first pulse (2-byte variable).
2) Measure and add the 6 intervals into the low byte of the sum. If there is overflow (STATUS,C=1) then increment the high byte.
3) Divide the 2-byte variable by 6. Actually you can first divide it by 2 (shift right one bit) and then divide by 3.

jak oddzielić połowę rejestru 8 bit

```
MAIN:
ldi r18, 0xAB //DANE 1010'1011
mov r19, r18
ldi r20, 0b11110000//maska do oddzeilenia 4 starszych bitów
and r19, r20

ror r19//przesuwamy w prawo by bity starsze były na miejscu młodszych
ror r19
ror r19
ror r19
//r19 ma starszą część danych (A)
```

```
mov r19, r18
ldi r20, 0b00001111//maska do oddzielenia 4 młodszych  bitów
and r19, r20
//r19 ma młodszą część danych (B)
```

LSL
LSR

czujnik inteligentny czyli taki, który ma w sobie blok akwizycji sygnałów i z tego czujnika wychodzi sygnał cyfrowy. Czyli on wchłania akwizycję sygnałów i w takim przypadku nie będzie zakłóceń w transmisji.